CHAPTER 14: A/B, SHADOW, AND NOT OVERCLAIMING

Summary: *Two things go wrong when a change is tested. The commitments that should have been locked before the run get decided afterwards, once somebody has seen which way the numbers went. And when quality drops in production, the team reaches for operational counters that are physically incapable of showing a quality problem. This chapter covers what has to be fixed in advance, how to get new evidence without betting the user base on it, and why a model that behaves differently today may be running identical weights.*

Two Things Get Locked Before the Experiment Runs

An experiment has exactly two commitments that must be made while the outcome is still unknown.

What **question is being asked**. The specific comparison, stated in advance, so it cannot be replaced later by whichever comparison happened to come out well.

How **big a sample it will take to answer** it. Fixed before a single measurement arrives.

Every other candidate for “settle this first” needs numbers that nothing has produced yet. You cannot set a threshold against results before the arms have served any traffic. You cannot pick a winner before there is anything to compare. A **prerequisite is something you could actually write down today**.

The reason the second commitment matters as much as the first is unglamorous: a sample size fixed in advance is what stops the run from ending the moment the numbers look good. Deciding when to stop after watching the numbers is deciding what the answer is.

One Difference Between the Arms, or the Result Says Nothing

There is a second way to spoil a comparison and it has nothing to do with sample size. An arm carrying **more than one change** cannot tell you **which change did anything**.

The shape is easy to describe and easy to walk into. A team wants to try a new prompt. While they are deploying it they also move that arm to a stronger model and let its retrieval reach further, because all three were on the list and shipping them together saves a cycle. The arm wins. Now nobody can say whether the prompt helped, whether it hurt and was carried by the model, or whether the whole gain came from retrieval depth and the prompt is dead weight they will maintain forever.

The **result is not merely imprecise, it is unattributable**, and no amount of extra traffic repairs it. That distinguishes this defect from an underpowered run, which more data does fix.

What makes it expensive is the disguise. The experiment produces a number, the number has a direction, and a direction feels like a finding. Meanwhile the traffic and the weeks that a clean comparison needed have been spent. A **confounded run costs the same as a good one** and **returns less than none**, because it also leaves behind a belief nobody has grounds for.

Three things that look like flaws in these scenarios are not. **Random assignment** is the mechanism making the arms comparable in the first place, and swapping it for assignment by geography or by any other existing attribute introduces precisely the systematic difference randomisation exists to remove. A **longer run** narrows the interval, and it spans more of the week-to-week movement any real traffic has in it. **Watching cost alongside quality** is what catches a variant that wins by spending more.

The **named anti-pattern is the bundled arm**: several improvements shipped to one variant together, on the reasoning that they are all improvements. They may all be improvements. The run will not tell you.

The 50-Session Winner

Name the failure: the **50-session winner** is a variant declared better on a sample too small to distinguish it from noise, usually because somebody was watching a dashboard and the line moved.

It is worth being precise about why this is not merely sloppy. Sample size is the lesser half of it. What does the damage is that the **stopping rule got chosen after somebody had already seen the data**, which converts an experiment into a hunt for the moment when the difference looked convincing. Given enough peeking, that moment arrives whether or not a real difference exists.

The **named anti-pattern is stopping when the result is pleasing**. It costs nothing at the time and produces a change that was never actually evidenced, which then becomes the baseline everybody compares to next.

Offline Evidence First, Then Capped Exposure

When a change needs validating, the strongest answers do two things at once, and each half rules out a different family of wrong answers.

Gather task-level evidence before exposing any traffic. This is the offline half. It catches regressions where they cost nothing.

Then **limit and monitor whatever exposure you do create**. This is the online half. It ensures that anything the offline suite missed lands on a small population rather than on everybody.

Between them, a regression surfaces either before release or inside a controlled slice, and **never across the whole user base at once**. Options that offer one half without the other fail in recognisable ways: pure offline validation adds no evidence from real traffic, and uncapped exposure adds no protection at all.

There is a further property that separates good online evidence from theatre. **Genuinely new evidence** has to arrive from **inputs your offline suite has never once encountered**. Running production traffic against tests you already passed is not new information, however reassuring the green dashboard is. The point of exposing anything is to encounter inputs nobody anticipated, which is precisely why the exposure has to be capped.

Counters Cannot See a Quality Regression

This is the diagnostic idea that decides the most items in the second half of this chapter.

When answer quality degrades, the only sources that can diagnose it are the ones showing what the **failing requests actually carried**: their prompts, the material retrieved for them, the responses they produced, the tool results they consumed.

The sources that cannot are **aggregate operational counters**, and the reason is structural rather than a matter of resolution. Request rate, error rate, latency percentiles, token throughput, uptime: every one of these **looks identical before and after a quality regression**. The system is still serving requests, still returning 200s, still responding in the same time. Nothing in an operational counter has any relationship to whether the answers are right.

So when a scenario asks which two sources would diagnose the problem, the pair is always content-bearing. Anything you could have **graphed before the incident and after it**, seeing the **same shape**, **cannot be the answer**.

Binding an Answer to Its Evidence

A specific requirement appears repeatedly and has exactly one kind of answer: making sure a response does not assert things its sources do not support.

Only a control on **what may appear in the output binds an answer to its evidence**.

The alternatives all do something real and none of them does this. **Detecting the problem** tells you it happened. **Restricting access** changes what the system can reach, not what it may claim about what it reached. **Sanitising the input** cleans what goes in, and the fabrication happens later.

That is the discriminating question to carry: does this **control act on what the answer is allowed to say**? If it acts anywhere else in the pipeline, it is doing a different job.

One documented technique deserves mention here because it is a genuine detection method rather than a constraint: **best-of-N verification** runs the **same prompt multiple times** and **compares the outputs**, on the basis that **inconsistencies** across them can indicate **hallucination**.[1] It sits alongside chain-of-thought

verification, iterative refinement, and restricting the model to the information in the supplied documents rather than its general knowledge. Note which of those is the constraint and which are the checks.

Model IDs Are Pinned. The Infrastructure Around Them Is Not.

This section explains a whole class of “the model changed” scenarios, and the resolution is usually that it did not.

Each **Claude model ID** identifies a **pinned version** whose **underlying model stays constant** for the lifetime of that ID.[2] That guarantee covers model IDs. It does **not** cover the convenience aliases the API accepts for some earlier models, where an alias resolves to the most recent dated snapshot for its minor version. An **alias is a moving target; an ID is not**.

From the 4.6 generation onward, IDs use a dateless format, and the documentation explicitly corrects the misconception this invites. A dateless ID is **not** an evergreen pointer to the latest or best-performing version. It is the **canonical ID for that release**, it **maps to a single fixed snapshot**, it is **never updated in place**, and an updated version ships under a **new model ID**.[2]

Now the part that resolves the confusing scenarios. **Model weights are fixed** for a given ID, but the **serving infrastructure** around the model is **not**. The request router, the safety classifiers and the sampling logic can all change over time, and an update to any of them can shift what you observe while the **model ID and the weights sit completely unchanged**.[2]

So the documented diagnosis: where **unexpected behavioural differences** appear on a **previously stable model ID**, an **infrastructure update** is the most likely cause. Not a silent model swap, which the versioning guarantee rules out. That distinction matters because the two have completely different responses: one is a real change to investigate and possibly re-baseline against, the other is a search for something that did not happen.

One related planning fact: Anthropic publishes which models are active, deprecated or retired with retirement dates and recommended replacements, and advises **testing thoroughly against replacement models well before the retirement date**, in

order to measure the replacement's performance on the tasks that actually matter to you.[3] A migration validated on generic capability is not a migration validated on your tasks.

What Batch Will Not Accept

Batch is the obvious vehicle for large evaluation and comparison runs, and a small set of parameters are unsupported. Including any of them returns a validation error, and each has a reason worth understanding rather than memorising blindly:[4]

Unsupported	**Why**
Streaming	Batch results come back as a single file, not a stream
The **fast-mode speed parameter**	It tunes synchronous latency, which does not apply to asynchronous processing
Stateful thread parameters	Batch requests are not stateful
Cache and context routing hints	They apply only to synchronous request scheduling

The pattern across all four: every one of them is a property of **synchronous, stateful, streaming interaction**, and batch is none of those things. If you can see why a parameter belongs to that world, you can predict it will be rejected without having memorised the list.

Checking Output Against Its Source, and Turning One Report Into a Rate

Two diagnosis patterns recur often enough to be worth setting out on their own, and both are about closing the distance between an anecdote and something you can act on.

Material that is not in the source. A summariser inserts a detail the document never contained. Anthropic's guidance on this names techniques that operate at two different moments, and the workable answer uses both.[5] Ahead of generation: **restrict the model to the supplied documents** and tell it to say so when they do not answer the question, which turns a silent invention into a stated absence. After generation: **check the claims in the output against that same material** automatically, then **route whatever scores badly to a person**. Without that last step the check is decorative: a score nobody routes on changes no output.

Each half alone leaves a hole. Instructions are followed most of the time, which is a different property from always. A check with no route attached notices the problem and ships it anyway.

The distractors here are worth naming because they share a shape: each changes how the model generates without ever comparing the output to the input. **Looser sampling** gives it more room to invent. **Summarising the summary** gives the second pass nothing but the first one output, so an invention travels forward while the prose around it grows tidier. **More output length** is unrelated to whether the writing is anchored, and offers more space for an unsupported claim.

One report, and the question of whether it matters. Somebody flags a single bad output on a system that has run for months. The team wants to know if this is an architecture problem or an unlucky Tuesday.

One report establishes that the failure is possible. It says nothing at all about frequency, and frequency is what decides the response. So the first move is arithmetic: **draw a sample of recent output and count**, which converts an anecdote into a rate. The second is structural: establish whether the **pipeline was ever built to catch this** at all. The same rate means different things depending on whether the relevant control is missing, present and misconfigured, or present and working inside limits somebody already accepted.

Both wrong directions decide before measuring. **Withdrawing the service** trades a risk of unknown size for a certain cost, and removes the traffic the investigation would have sampled. **Filing it as a one-off** is not a finding about how rare it is; it is the absence of one.

The **named anti-pattern is the diagnosed anecdote**: a single observation treated as a measurement, in either direction. Concluding the system is broken and concluding it is fine are the same error made with the same evidence.

How Wrong Answers Are Built on Testing and Diagnosis Questions

- A **prerequisite that needs the results**. The option fixes a threshold or a decision against measurements that do not exist yet. The test: could this be **written down before any traffic is served**?
- **Validation with no new information, or no protection**. The option runs more offline tests, or exposes everybody. The test: does this bring in **inputs the suite has not seen**, and does it **cap who is affected** if it goes wrong?
- **Operational counters offered against a quality problem**. The test: would this metric have **looked different before the regression**?
- A **control at the wrong stage of the pipeline**. The option detects, restricts access, or sanitises input where the requirement is that the answer not overclaim. The test: does this constrain what the **output may say**?
- A **model swap assumed**. The test: did the **model ID change**, or **only the behaviour**?

When the scenario says	The answer is usually	The trap is usually
An experiment is being planned, two commitments wanted	Which question it answers, and how big a sample that takes	Thresholds or decision rules that require results to exist first
A variant looks better after a short run	The stopping rule must be fixed in advance	Declaring a winner because the line moved

When the scenario says	The answer is usually	The trap is usually
A change needs validating before rollout	Task-level evidence offline, then capped and monitored exposure	Offline alone, which adds nothing new, or full rollout, which protects nobody
Online evidence is being gathered	It has to arrive from inputs the offline suite never encountered	Replaying tests already passed against production traffic
Answer quality regressed, two diagnostic sources wanted	Whatever shows what the failing requests carried: prompts, retrievals, responses, tool results	Request rate, error rate, latency or throughput, which look identical either side
Responses must not assert unsupported things	A control on what may appear in the output	Detection, access restriction, or input sanitisation
Hallucination must be detected rather than prevented	Run the prompt several times and compare for inconsistency	Treating a detector as a constraint
Behaviour changed on a stable model ID	An infrastructure update: router, classifiers or sampling logic	Assuming the weights or the model were silently swapped
A dateless model ID is in use	It is a fixed snapshot; updates ship as a new ID	Treating it as an evergreen pointer to the latest version

When the scenario says	The answer is usually	The trap is usually
A convenience alias is in use	It resolves to the most recent dated snapshot for its minor version	Assuming the pinning guarantee covers aliases
A model is being retired	Test against the replacement on your own tasks, well before the date	Migrating on generic capability claims
A batch job is rejected	Streaming, fast-mode speed, stateful thread and routing hint parameters are unsupported	Assuming any Messages request can be batched unchanged
A new prompt is tested against the current one in production	Change one thing in the arm and nothing else	A model upgrade and deeper retrieval shipped into the same arm alongside it
An agent fails a minority of runs with no pattern in the output	Compare the step-by-step records of failing and passing runs on similar inputs	A larger model, more evaluation cases, or a rerun at different sampling settings
A summariser introduces detail the source never contained	Restrict it to the documents, verify claims against them, and route poor scores to a person	Higher temperature, a second summarising pass, or a longer output allowance

When the scenario says	The answer is usually	The trap is usually
One user reports one bad output on a long-running system	Sample recent output to get a rate, and establish whether a control was ever meant to catch it	Withdrawing the service immediately, or recording it as an isolated case

The habit worth carrying: before running anything, write down **what result would make you abandon the change**. An experiment with no losing outcome is a launch with extra steps.

What to Remember

- **Two commitments are locked** before an experiment runs: which **question it answers**, and how **big a sample that will take**. Everything else needs **numbers that nothing has produced** yet. A prerequisite is **something you could write down today**.
- The **50-session winner** is a variant declared better on a **sample too small to separate it from noise**. The defect is not the small number, it is that the **stopping rule was chosen after seeing the data**, which turns an experiment into a **hunt for a convincing moment**.
- **Validate offline first**, then **cap and monitor the exposure you create**, so a regression is caught **either before release or on a small population**, never by the **whole user base at once**. Offline alone **adds no new information**; uncapped rollout **adds no protection**.
- Genuinely new online evidence arrives from **inputs the offline suite has never once encountered**. Replaying passed tests against production traffic is **reassurance, not evidence**.
- Only sources showing what the **failing requests actually carried** can diagnose a quality regression: their **prompts, retrieved material, responses and tool results**. **Aggregate operational counters look identical before and after**, because request rate, error rate, latency and throughput have **no relationship to whether answers are right**.

- Only a control on what may **appear in the output** binds an answer to its evidence. Detecting the problem, restricting access and sanitising input each do something real and **none** of them does this.
- **Best-of-N verification** runs the same prompt repeatedly and compares outputs, treating **inconsistency as a signal of hallucination**. It is a **detector, not a constraint**, and sits alongside chain-of-thought verification, iterative refinement, and restricting the model to the supplied documents.
- A model ID is a **pinned version** whose underlying model **stays constant for that ID's lifetime**. The guarantee does **not cover convenience aliases**, which resolve to the **most recent dated snapshot** for their minor version.
- A dateless ID from 4.6 onward is **not an evergreen pointer**. It is the **canonical ID for that release**, mapped to **one fixed snapshot**, **never updated in place**, and an updated version **ships under a new model ID**.
- **Weights are fixed for an ID**; the serving infrastructure is **not**. The request router, safety classifiers and sampling logic can change, and an update to any of them can **shift observed behaviour** with the ID and the weights untouched. Unexpected behaviour on a previously stable ID points at an **infrastructure update**, not a silent model swap.
- **Test against a replacement model well before the retirement date**, on the **tasks that matter** to you. Generic capability claims are **not a migration test**.
- Batch rejects **streaming**, the **fast-mode speed parameter**, **stateful thread parameters**, and **cache and context routing hints**, all with a **validation error**, because each belongs to **synchronous, stateful or streaming interaction** and **batch is none** of those.